CHAPTER 3

Neural Networks

Neural networks are function approximators built from many simple units. Each unit computes a weighted sum of its inputs and passes it through a non-linear function. Stacked in layers, these units can represent remarkably complex relationships, from recognising handwriting to translating languages.

3.1 The Perceptron

The perceptron, introduced by Frank Rosenblatt in 1958, is the simplest neural unit. It receives an input vector x, multiplies each component by a weight, adds a bias b, and outputs 1 if the result is positive and 0 otherwise. Geometrically, the weights define a hyperplane that splits the input space into two half-spaces: one for each class.

$$y = \mathrm{step}(w \cdot x + b)$$

The weights w and the bias b are learned from labelled examples. The perceptron learning rule nudges the weights toward misclassified positive examples and away from misclassified negative ones. If the two classes can be separated by a straight line (or hyperplane), this procedure is guaranteed to converge.

However, a single perceptron cannot solve problems that are not linearly separable. The classic counter-example is XOR: no single line separates the points (0,1) and (1,0) from (0,0) and (1,1). This limitation, highlighted by Minsky and Papert in 1969, motivated networks with hidden layers.

Key idea

> *A perceptron is a linear classifier. Depth (hidden layers) plus non-linearity is what gives neural networks their expressive power.*

3.2 Multi-layer Networks

A multi-layer perceptron (MLP) chains layers of units. The first layer receives the input, one or more hidden layers transform it, and the output layer produces the prediction. Figure 3.1 shows a network with three inputs, four hidden units and one output.

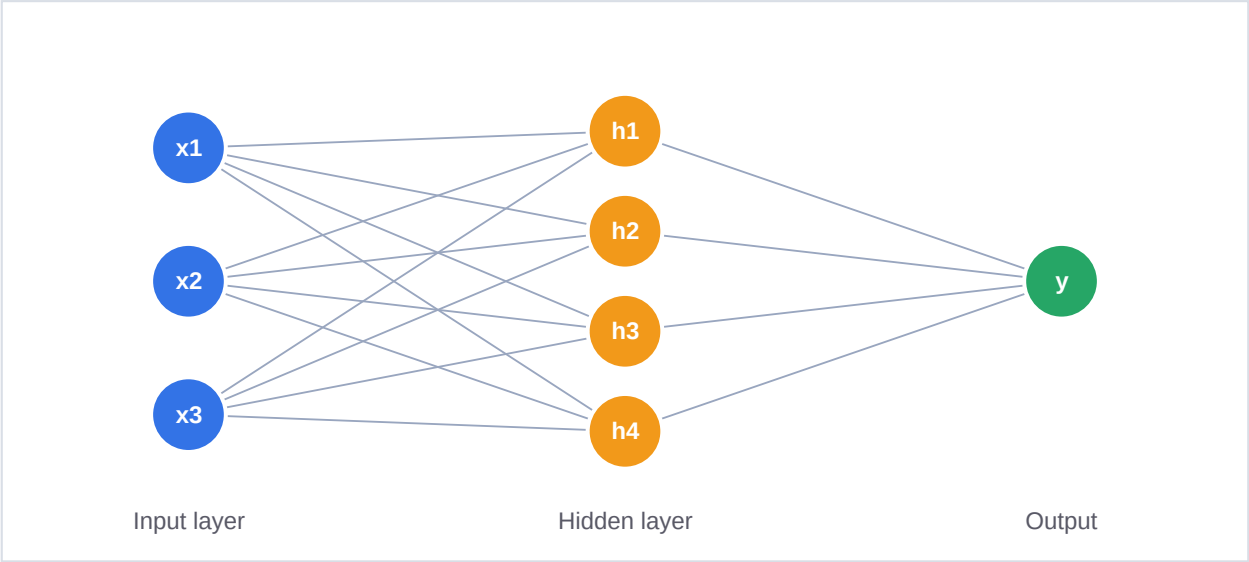

Figure 3.1 — A fully connected network with one hidden layer. Every arrow carries a learnable weight.

3.3 Activation Functions

Without a non-linear activation, stacking layers would collapse into a single linear map. The sigmoid function squashes values into (0, 1) and was popular historically, but its gradient vanishes for large inputs. The rectified linear unit, ReLU(z) = max(0, z), is cheap to compute and keeps gradients alive for positive inputs, which is why it became the default choice in deep networks.

$$\mathrm{sigmoid}(z) = 1 / (1 + e^{-z}) \qquad \mathrm{ReLU}(z) = \max(0, z)$$

3.4 Training with Gradient Descent

Training means choosing weights that make the network's predictions match the labels. We measure the mismatch with a loss function L, for example the mean squared error or the cross-entropy. Gradient descent then repeatedly moves every weight a small step against the gradient of the loss:

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

The learning rate eta controls the step size. If it is too large the updates overshoot the minimum and the loss oscillates or diverges; if it is too small training crawls and may stall on plateaus. In practice the learning rate is the single most important hyper-parameter to tune, and schedules that decrease it over time are common.

Computing the gradient on the full dataset is expensive. Stochastic gradient descent (SGD) estimates it from a small random mini-batch instead. The noise this introduces is not only cheaper, it also helps the optimiser escape shallow local minima.

Worked example

> Suppose a weight w = 0.80, the gradient dL/dw = 0.50 and eta = 0.10. The update gives w = 0.80 - 0.10 * 0.50 = 0.75. The weight moved slightly in the direction that reduces the loss.

3.5 Backpropagation

Backpropagation is the algorithm that computes dL/dw for every weight efficiently. It applies the chain rule of calculus layer by layer, starting at the output and moving backward. Each layer receives the gradient of the loss with respect to its outputs, multiplies it by its local derivative, and passes the result to the layer before it.

$$dL/dw1 = dL/dy * dy/dh * dh/dw1$$

Because intermediate results from the forward pass are reused, backpropagation costs roughly the same as one forward pass. This efficiency is what makes training networks with millions of parameters practical.

Two classic difficulties appear in deep networks. Vanishing gradients occur when many small derivatives are multiplied together, so early layers barely learn. Exploding gradients are the opposite problem. ReLU activations, careful weight initialisation, normalisation layers and residual connections all mitigate these issues.

Review Questions

1. Why can a single perceptron not learn the XOR function? Sketch the four points to justify your answer.

2. What would happen to a deep network if every activation function were the identity f(z) = z?

3. Compare sigmoid and ReLU. Give one advantage of each.

4. A training loss oscillates wildly from step to step. Which hyper-parameter would you change first, and how?

5. Explain in your own words why backpropagation is efficient.

Further reading

Goodfellow, Bengio and Courville, Deep Learning (MIT Press, 2016), chapters 6-8.